

```

import pandas as pd
import numpy as np
import re
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.pipeline import Pipeline
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download required NLTK data
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)

def preprocess_text(text):
    """Clean and preprocess text"""
    # Lowercase and remove HTML/special chars
    text = re.sub(r'<.*?>', '', text.lower())
    text = re.sub(r'^a-zA-Z\s', '', text)
    text = ' '.join(text.split())

    # Tokenize and remove stopwords
    stop_words = set(stopwords.words('english'))
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words and len(word) > 2]

    return ' '.join(tokens)

def load_sample_data():
    """Create sample IMDb-like data"""
    reviews = [
        ("This movie was absolutely fantastic! Great acting and storyline.", 1),
        ("Terrible film, waste of time. Poor acting and boring plot.", 0),
        ("Amazing cinematography and brilliant performances.", 1),
        ("One of the worst movies I've ever seen. Disappointing.", 0),
        ("Excellent direction and compelling narrative. Recommended!", 1),
        ("Boring and predictable. Nothing special about this film.", 0),
        ("Outstanding movie with incredible character development.", 1),
        ("Poor script and mediocre acting. Not worth watching.", 0),
        ("Brilliant storyline with unexpected twists. Loved it!", 1),
        ("Disappointing ending and weak characters.", 0),
        ("The special effects were mind-blowing and engaging.", 1),
        ("Couldn't finish watching. Extremely boring.", 0),
        ("Masterpiece with outstanding performances.", 1),
        ("Waste of money and time. Terrible in every aspect.", 0),
        ("Captivating from start to finish. Brilliant acting.", 1),
        ("Poor dialogue and unconvincing plot. Disappointing.", 0),
        ("One of the best films of the year. Entertaining.", 1),
        ("Confusing plot and poor execution. Not recommended.", 0),
        ("Exceptional storytelling and beautiful cinematography.", 1),
        ("Boring and clichéd. Nothing new or interesting.", 0)
    ]
    return pd.DataFrame(reviews, columns=['review', 'sentiment'])

```

```

def train_sentiment_classifier(df):
    """Train logistic regression model"""
    # Preprocess text
    df['cleaned_review'] = df['review'].apply(preprocess_text)

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        df['cleaned_review'], df['sentiment'],
        test_size=0.2, random_state=42, stratify=df['sentiment']
    )

    # Create pipeline with TF-IDF and Logistic Regression
    model = Pipeline([
        ('tfidf', TfidfVectorizer(
            max_features=5000,
            ngram_range=(1, 2),
            min_df=2,
            max_df=0.8
        )),
        ('classifier', LogisticRegression(random_state=42, max_iter=1000))
    ])

    # Train model
    model.fit(X_train, y_train)

    # Make predictions
    y_pred = model.predict(X_test)

    return model, X_test, y_test, y_pred

def evaluate_model(y_test, y_pred):
    """Evaluate model performance"""
    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print("="*40)
    print("MODEL EVALUATION")
    print("="*40)
    print(f'Accuracy: {accuracy:.4f}')
    print(f'F1-Score: {f1:.4f}')
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=['Negative', 'Positive']))

    return accuracy, f1

def predict_sentiment(model, text):
    """Predict sentiment for new text"""
    cleaned_text = preprocess_text(text)
    prediction = model.predict([cleaned_text])[0]
    probability = model.predict_proba([cleaned_text])[0].max()

    sentiment = "Positive" if prediction == 1 else "Negative"
    return sentiment, probability

# Main execution
def main():
    print("Loading sample IMDb data...")

```

```
df = load_sample_data()
print(f"Dataset: {len(df)} reviews")
print(f"Distribution: {df['sentiment'].value_counts().to_dict()}")

print("\nTraining model...")
model, X_test, y_test, y_pred = train_sentiment_classifier(df)

print(f"Test set size: {len(X_test)}")

# Evaluate
accuracy, f1 = evaluate_model(y_test, y_pred)

# Test examples
print("\n" + "="*40)
print("TESTING NEW EXAMPLES")
print("="*40)

test_reviews = [
    "This movie is absolutely amazing!",
    "Terrible and boring film.",
    "Outstanding performance by all actors!"
]

for review in test_reviews:
    sentiment, confidence = predict_sentiment(model, review)
    print(f"'{review}' → {sentiment} ({confidence:.3f})")

if __name__ == "__main__":
    main()

# df = pd.read_csv('imdb_dataset.csv') # columns: 'review', 'sentiment'
```